

Figure 4: Initial steps of sorting the array

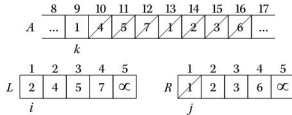


Figure 5: Initial steps of sorting the array

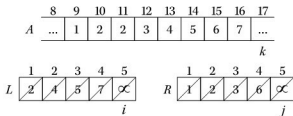


Figure 6: The final output

programs) and use the simple algorithms as sub-routine calls in your main algorithm (depending on the model you opt for).

Now we can look at another simple algorithm that can handle errors. The following code shows the multiplication of matrices:

```
MATRIX-MULTIPLICATION(A, B)
if columns[A] != rows[B]
    then error "This operation is not allowed"
else for i ← 1 to rows[A]
    do for j ← 1 to columns[B]
        do C[i, j] ← 0
        for p ← 1 to columns[A]
            do C[i, j] ← C[i, j] + A[i, p] · B[p, j]
return C
```

Before we move on to the complex stuff, you can test yourself by considering the following problem (Table 1). You are required to find out the maximum value of n that corresponds to each of the stipulated times. The data provided along with it is that you have $f(n)$ in milliseconds. Try solving it!

We can see that the style remains the same when we try to write algorithms in C. If we need to compare functions for a pointer to an integer, we can have the following:

```
#include "compare.h"

int int_is_equal(void *vplace1, void *vplace2)
{
    int *place1;
```

```
int *place2;

place1 = (int *) vplace1;
place2 = (int *) vplace2;

return *place1 == *place2;
}

int int_is_compare(void *vplace1, void *vplace2)
{
    int *place1;
    int *place2;

    place1 = (int *) vplace1;
    place2 = (int *) vplace2;

    if (*place1 < *place2) {
        return -1;
    } else if (*place1 > *place2) {
        return 1;
    } else {
        return 0;
    }
}
```

You may find a reference to a header file in the program. So you need `compare.h` along with it to get the desired result:

```
#ifndef ALGORITHM_COMPARE_INT_H
#define ALGORITHM_COMPARE_INT_H

#ifdef __cplusplus
extern "C" {
#endif

int int_is_equal(void *place1, void *place2);

int int_is_compare(void *place1, void *place2);

#ifdef __cplusplus
}
#endif

#endif
```

There could be problems that belong to the *NP-complete* category. These problems may not have an exact solution. We will be discussing these aspects once we are done with topics like asymptotic notations and complex algorithms. 

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com